JAVA PROGRAMMING

Chapter 1: Introduction to Programming

Personal Notes

Before jumping into Java syntax, it helps to understand what programming actually is, the different families of programming languages, and how Java fits into the picture. This chapter covers the big-picture ideas every Java learner should know before writing serious code.

These topics also appear directly in interview questions. Knowing what stack vs heap means, or what garbage collection does, can save you in a viva or technical round.

1. What is Programming?

Programming is giving step-by-step instructions to a computer. Think of it like explaining a recipe to a cook. If you say "take milk, add sugar, and boil," you get tea. A program does the same thing — it tells the computer exactly what to do, step by step.

Without instructions, a computer is just an empty notebook. It does nothing on its own. The moment you give it a program, it can calculate, decide, and produce results.

In Java, we write these instructions in a language that humans can read, and the computer translates it later into something the machine can understand. For example, if you tell Java to take two numbers, add them, and print the result, that is exactly what happens.

```
int a = 10;
int b = 20;
int sum = a + b;
System.out.println(sum);   // prints 30
```

Just like a recipe produces food, a program produces an output. That is the simplest way to think about programming.

2. Types of Programming Languages

Computers only understand 0s and 1s. Writing in pure 0s and 1s is painful for humans, so we use programming languages as a bridge between human thinking and machine code.

Languages are divided based on how close they are to the machine:

- Low-level languages: very close to machine code, hard for humans to read (like Assembly)
- High-level languages: closer to English, easy for humans (like Java, Python, C++)

High-level languages are then translated to 0s and 1s by a compiler or interpreter. Java is a high-level language — you write something that looks almost like English, and Java handles the rest.

High-level languages are further grouped by their style of programming. The main three are:

2.1 Procedural Languages

In procedural programming, you tell the computer what to do step by step, one instruction after another. The order matters. It is like cooking — first boil water, then add sugar, then add tea leaves. If you swap the steps, you get a different result.

You break the program into small reusable blocks called functions, and combine them to build bigger tasks. C is the classic example of a procedural language.

Examples: C, Pascal, BASIC

2.2 Functional Languages

Functional programming is based on mathematical functions. The same input always produces the same output. 2 + 3 is always 5, no matter how many times you compute it.

Think of a vending machine. Insert the same coin and press the same button, and you always get the same snack. There are no hidden side effects.

Examples: Haskell, Scala, Lisp

2.3 Object-Oriented Languages (OOP)

Object-oriented programming is built around real-world things called objects. A car, for example, has properties (color, speed, price) and actions (drive, stop, honk). In OOP, you model these as objects in code.

This makes programming feel natural because you think in terms of real things instead of just steps or numbers. Java is primarily object-oriented, which is why it works so well for apps, games, and large projects.

Examples: Java, C++, C#, Python (also supports OOP)

> **Tip:** In Java, almost everything you write lives inside a class. Even your main method is inside a class. That is OOP at work — you cannot escape it.

3. Mixed-Style Languages

Some languages are flexible and let you use more than one style. Python is the best example — you can write it like a step-by-step recipe (procedural) or build it around objects (OOP). The choice is yours.

Java is mostly object-oriented, but you can still write small procedural-feeling programs in it. This flexibility makes such languages useful for many different kinds of problems.

Style	Core Idea	Example Languages
Procedural	Step-by-step instructions in order	C, Pascal
Functional	Pure functions, same input → same	Haskell, Scala

Style	Core Idea	Example Languages
	output	
Object-Oriented	Real-world objects with data and actions	Java, C++
Mixed-Style	Supports more than one style	Python, JavaScript

4. Static vs Dynamic Languages

This is one of the most important differences for any beginner to understand, especially because it changes how you write code daily.

Static Languages (like Java)

In static languages, every variable must be declared with a type before use. The type is fixed and cannot change.

```
int age = 25;       // age is an int forever
age = "twenty-five"; // ERROR — cannot assign text to an int
```

This is stricter, but it catches bugs early. The compiler refuses to run the program if types do not match.

Dynamic Languages (like Python)

In dynamic languages, you do not declare types. A variable can hold any kind of value, and you can even change the kind later.

```
# Python
age = 25
age = "twenty-five"   # totally fine
```

This is more flexible, but mistakes can hide until the program is running.

> **Analogy:** Static = wearing a school uniform (strict, same rules, fewer surprises). Dynamic = casual clothes (more freedom, but sometimes messy).

Static (Java)	Dynamic (Python)
Types declared before use	No type declaration needed
Errors caught at compile time	Errors usually caught at runtime
Stricter, safer for large projects	More flexible, faster to prototype
Examples: Java, C, C++	Examples: Python, JavaScript, Ruby

5. Errors in Programming

Mistakes are a normal part of writing code. Even experienced developers make them every day. The trick is knowing which kind of error you are dealing with — that tells you where to look.

5.1 Syntax Errors

Syntax errors are grammar mistakes in your code. The most common one in Java is forgetting a semicolon.

```
int x = 10   // missing semicolon → syntax error
```

The compiler refuses to compile the program until you fix it. These are easy to catch because the compiler points right at the line.

5.2 Type Errors

Type errors happen when you use the wrong kind of data — like trying to put text inside an integer.

```
int age = "twenty";   // type error → cannot assign String to int
```

5.3 Runtime Errors

Runtime errors happen while the program is actually running. The code compiled fine, but something went wrong during execution. The classic example is dividing by zero.

```
int a = 10;
int b = 0;
int c = a / b;   // crashes at runtime → ArithmeticException
```

> **Important:** Read error messages carefully. They almost always tell you the file, the line number, and the kind of error. Beginners often panic at red text — but that text is your best friend when debugging.

6. Memory in Java — Stack vs Heap

When a Java program runs, it stores data in two main memory areas: the Stack and the Heap. Knowing the difference is important because it explains why some things behave the way they do in Java.

6.1 The Stack

The stack is like a scratch notepad you use while solving a math problem. It holds short-lived things: method calls, local variables, and primitive values.

Each method gets its own little space on the stack. When the method finishes, that space is automatically thrown away. The stack is fast but small.

6.2 The Heap

The heap is like a large cupboard where you store bigger, longer-lasting things — namely, objects. When you write new Car() or new Student(), that object goes into the heap.

The heap is bigger than the stack but slower to access. Java manages it for you automatically.

Quick Diagram

```
     STACK                        HEAP
+-----------------+      +-------------------------+
| main()          |      |                         |
|   age = 25      |      |   Car object            |
|   myCar  ---------->   |   color = "red"         |
|                 |      |   speed = 80            |
+-----------------+      +-------------------------+
  (short-lived,           (long-lived,
   primitives,             objects live here)
   references)
```

In the diagram, age = 25 sits directly on the stack because int is a primitive. But myCar is a reference variable — it sits on the stack, but it points to the actual Car object stored on the heap.

Stack	Heap
Stores primitives and references	Stores actual objects
Short-lived (cleared when method ends)	Long-lived (cleared by garbage collector)
Fast access	Slower access
Smaller in size	Larger in size

7. Objects and Reference Variables

In Java, an object represents a real thing — a car, a book, a student. To actually use an object, you need a reference variable. The reference variable acts like a remote control: the object sits in memory, and the reference lets you control it.

Look at this line:

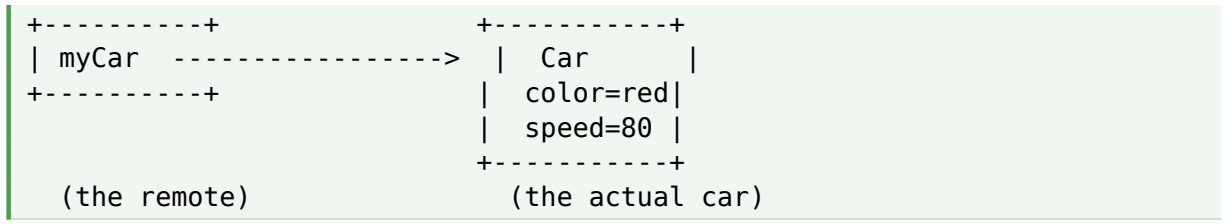
```
Car myCar = new Car();
```

Two things are happening here:

- new Car() creates the actual Car object in the heap
- myCar is the reference variable, sitting on the stack, that points to that object

Visual Idea

```
   STACK                          HEAP
```



Without the reference, you cannot control the object. Without the object, the reference points to nothing. Both are needed.

> **Remember:** A reference variable is NOT the object itself. It only holds the address of the object. This is why two references can point to the same object — like two remotes for one TV.

8. Garbage Collection

When an object is no longer being used by any reference, Java automatically removes it from memory. This process is called Garbage Collection (GC). You do not have to clean up memory manually like in older languages such as C or C++.

Think of it like your wardrobe. When you stop wearing certain clothes, eventually you throw them out or donate them to make space for new ones. Java does the same thing with unused objects — it keeps your program clean and prevents memory from filling up.

Example

```
Car myCar = new Car();
myCar = null;   // the Car object now has no reference
                // → garbage collector will remove it later
```

Once myCar = null is set, no one is pointing to the original Car object anymore. The garbage collector eventually notices this and frees that memory.

> **Why this matters:** Garbage collection is one of the biggest reasons Java is considered safer and easier than C or C++. You focus on writing logic; the JVM handles memory.

9. Quick Recap

A summary of everything in this chapter:

- Programming is giving step-by-step instructions to a computer
- Languages are grouped by style — procedural, functional, object-oriented
- Java is primarily object-oriented but supports multiple styles
- Static languages (Java) need declared types; dynamic languages (Python) do not
- Errors come in three flavors: syntax, type, and runtime
- Java uses two memory areas: stack (short-lived) and heap (long-lived objects)
- Reference variables on the stack point to actual objects on the heap

- Garbage collection frees unused objects automatically

10. What's Next?

Now that the big picture is clear, the next logical steps in your Java learning journey are:

- Setting up Java and writing your first program
- Understanding the structure of a Java program (class, main method)
- Variables and data types
- Operators and expressions
- Conditionals and loops

> **Key Takeaway:** You don't need to memorize every detail of stack vs heap or garbage collection right now. Just understand the idea. As you write more Java code, these concepts will start to make sense naturally — and one day they will save you in a tricky bug or an interview question.